POLITECNICO
MILANO 1863

Ingegneria del software

Design pattern comportamentali

Observer

Motivazioni

- Si assuma si voglia creare una applicazione per manipolare fogli di calcolo
- Una classe Spreadsheet contiene i dati e le formule inserite dall'utente
- Una classe BarChart permette di visualizzare i dati in un diagramma a istogrammi. Un'altra classe LineChart mostra i dati con una curva su un piano cartesiano.

Motivazioni

- **Problema:** gestire le interazioni tra una sorgente di dati (Spreadsheet) e multiple classi che utilizzano questi dati (diagrammi)
- I diagrammi non devono avere una dipendenza forte con la sorgente di dati (*observable*)
- La sorgente di dati non deve avere dipendenze forti con gli utilizzatori (*observers*)
- Modello publish-subscribe

Una soluzione non ideale

```
class Spreadsheet {
        private BarChart barChart;
        private LineChart lineChart;
        private float data[][];
        private void updateData(int x, int y, float value){
                data[x][y] = value;
                barChart.updateBar();
                lineChart.updateLine();
        }
        …
        public float[][] getData(){
                return data;
        }
}
```

Una soluzione non ideale

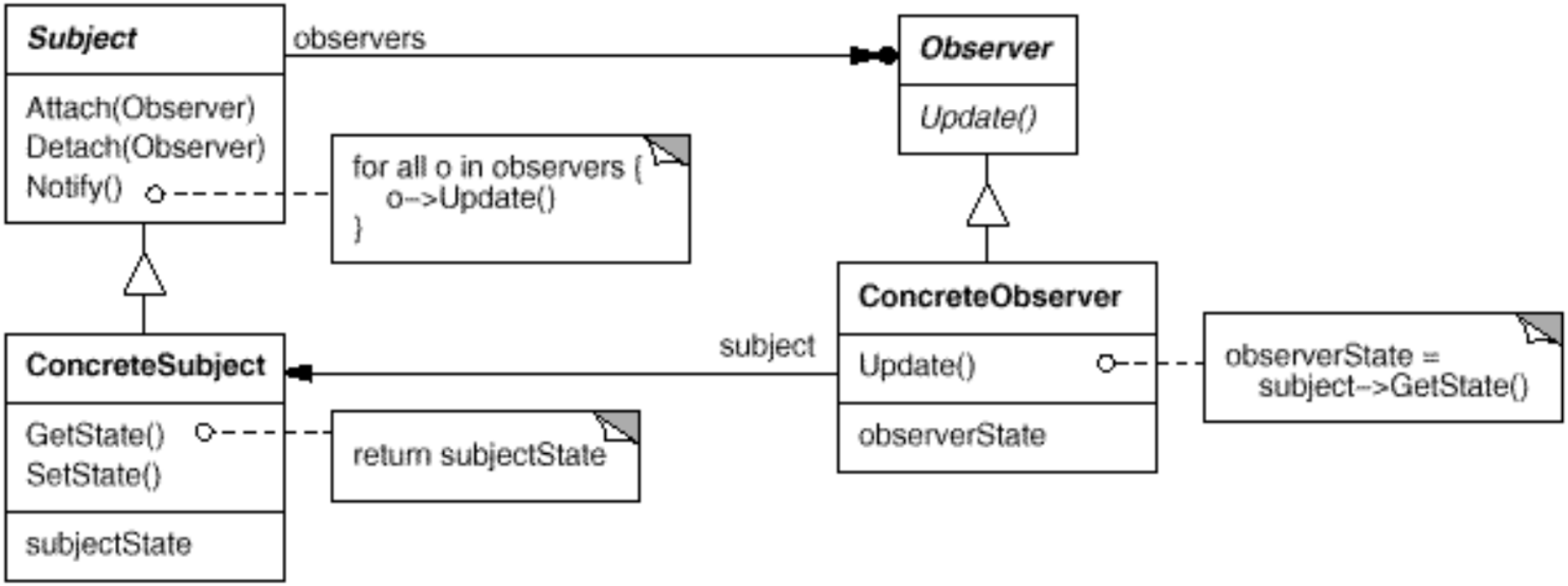
```
class BarChart {
        private Spreadsheet sheet;
        public void updateBar(){
                float data[][] = sheet.getData();
                …
                draw();
        }

        private void draw(){ … }
}
```

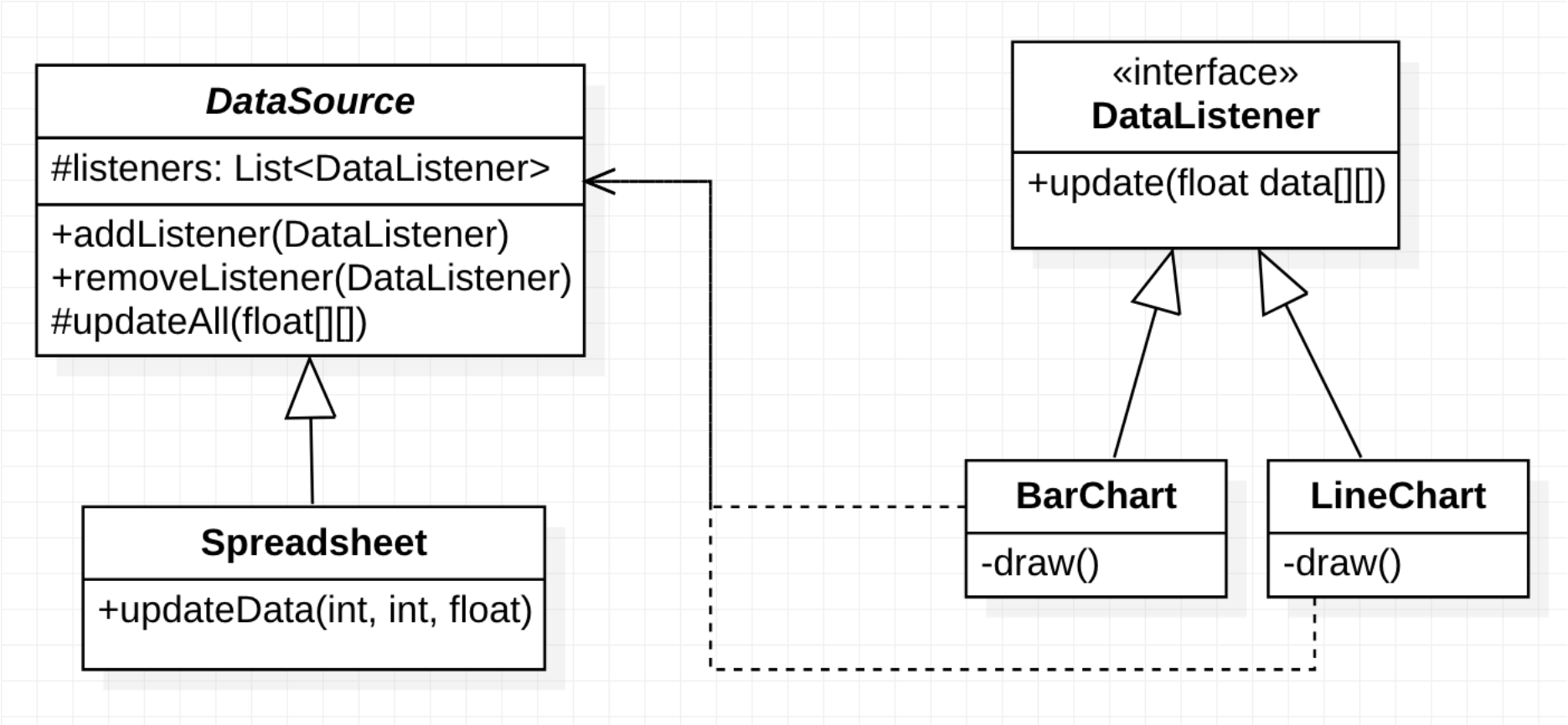
Obbiettivi

- Definire un sistema astratto per la comunicazione one-to-many
- Creare una classe astratta Observable, per rappresentare la sorgente di dati, che contiene una lista di Observer
- Creare una interfaccia Observer per rappresentare un utilizzatore dei dati
- Gli oggetti Observer possono iscriversi/disiscriversi agli aggiornamenti di Observable attraverso appositi metodi

Struttura


Subject
Attach(Observer)
Detach(Observer)
Notify()
observers
Observer
Update()
for all o in observers {
o->Update()
}
ConcreteSubject
GetState()
SetState()
subjectState
return subjectState
subject
ConcreteObserver
Update()
observerState
observerState =
subject->GetState()

Esempio


DataSource
#listeners: List<DataListener>
+addListener(DataListener)
+removeListener(DataListener)
#updateAll(float[][])
«interface»
DataListener
+update(float data[][])
Spreadsheet
+updateData(int, int, float)
BarChart
-draw()
LineChart
-draw()

Esempio

```
abstract class DataSource {
      List<DataListener> listeners = new ArrayList<>();
      public addListener(DataListener ld) {
            listeners.add(ld);
      }

      public removeListener(DataListener ld) {
            listeners.remove(ld);
      }

      protected updateAll(float data[][]){
            for(DataListener ld : listeners)
                  ld.update(data);
      }
}
```

Esempio

```
class Spreadsheet extends DataSource {
      private float data[][];
      …
      private void updateData(int x, int y, float value){
            data[x][y] = value;
            this.updateAll(data);
      }
}
```

Esempio

```
interface DataListener {
      update(float data[][]);
}

class BarChart implements DataListener {
      public void update(float data[][]){
            …
            draw();
      }

      private void draw(){ … }
}
```